```
!pip -q install rdkit
```

```
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 36.7/36.7 MB 30.4 MB/s eta 0:00:00
```

```
import numpy as np
import pandas as pd
import random
import matplotlib.pyplot as plt

from rdkit import Chem, DataStructs
from rdkit.Chem import AllChem

from sklearn.model_selection import StratifiedKFold, train_test_split, learning_curve, validation_curve
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.pipeline import Pipeline
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, roc_auc_score, roc_curve, auc

SEED = 42
N_SPLITS = 5

def set_seed(seed=42):
    random.seed(seed)
    np.random.seed(seed)

set_seed(SEED)
```

```
# Veri kümesini yükle
df = pd.read_csv("https://raw.githubusercontent.com/McahitKutsal/hivcsv/main/HIV7.csv")
df = df.dropna(subset=["smiles", "HIV_active"]).reset_index(drop=True)

print("Shape:", df.shape)
print(df["HIV_active"].value_counts())
df.head()
```

```
Shape: (2885, 4)
HIV_active
1    1473
0    1412
Name: count, dtype: int64
```

	Unnamed: 0	smiles	activity	HIV_active
0	0	CNC(=O)Nc1ccncc1	CI	0
1	1	O=C1NC(O)c2cccc(Cl)c2O1	CI	0
2	2	O=S1(=O)C=Cc2ccccc2CC1	CI	0
3	3	Cc1ccc(OS(=O)(=O)c2ccccc2[N+](=O)[O-])cc1	CM	1
4	4	CN(c1ccccc1)S(=O)(=O)c1cccc([N+](=O)[O-])c1	CM	1

```
def smiles_to_morgan(smiles, radius=2, nBits=2048):
    mol = Chem.MolFromSmiles(smiles)
    if mol is None:
        return np.zeros((nBits,), dtype=np.float32)
    fp = AllChem.GetMorganFingerprintAsBitVect(mol, radius, nBits)
    arr = np.zeros((nBits,), dtype=np.float32)
    DataStructs.ConvertToNumpyArray(fp, arr)
    return arr

X = np.array([smiles_to_morgan(s) for s in df["smiles"]], dtype=np.float32)
y = df["HIV_active"].astype(int).values

print("X shape:", X.shape, "| y shape:", y.shape)
```

```
[09:22:12] DEPRECATION WARNING: please use MorganGenerator
[09:22:12] DEPRECATION WARNING: please use MorganGenerator
[09:22:12] DEPRECATION WARNING: please use MorganGenerator
```

```
skf = StratifiedKFold(n_splits=N_SPLITS, shuffle=True, random_state=SEED)

fold_results = []
roc_data = []

for fold, (train_idx, test_idx) in enumerate(skf.split(X, y), start=1):
    print(f"==================== FOLD {fold} ====================")
    set_seed(SEED + fold)

    X_train_full, X_test = X[train_idx], X[test_idx]
    y_train_full, y_test = y[train_idx], y[test_idx]

    metrics = run_fold(X_train_full, y_train_full, X_test, y_test, fold_seed=SEED + fold)

    roc_data.append((metrics["roc_fpr"], metrics["roc_tpr"]))
    metrics_row = {k: v for k, v in metrics.items() if k not in ["roc_fpr", "roc_tpr"]}
    metrics_row["fold"] = fold
    fold_results.append(metrics_row)

    printable = {k: round(v, 4) if isinstance(v, float) else v for k, v in metrics_row.items()}
    print(printable)

results_df = pd.DataFrame(fold_results)
results_df = results_df[["fold", "accuracy", "precision", "recall", "f1", "roc_auc", "threshold"]]
results_df
```

```
==================== FOLD 1 ====================
{'accuracy': 0.8267, 'precision': 0.9042, 'recall': 0.7381, 'f1': 0.8127, 'roc_auc': np.float64(0.8938), 'thresho
==================== FOLD 2 ====================
{'accuracy': 0.8562, 'precision': 0.9271, 'recall': 0.7789, 'f1': 0.8466, 'roc_auc': np.float64(0.9142), 'thresho
==================== FOLD 3 ====================
{'accuracy': 0.8215, 'precision': 0.8871, 'recall': 0.7458, 'f1': 0.8103, 'roc_auc': np.float64(0.8926), 'thresho
==================== FOLD 4 ====================
{'accuracy': 0.8596, 'precision': 0.9115, 'recall': 0.8034, 'f1': 0.8541, 'roc_auc': np.float64(0.9234), 'thresho
==================== FOLD 5 ====================
{'accuracy': 0.8388, 'precision': 0.8769, 'recall': 0.7966, 'f1': 0.8348, 'roc_auc': np.float64(0.9124), 'thresho
```

	fold	accuracy	precision	recall	f1	roc_auc	threshold
0	1	0.826690	0.904167	0.738095	0.812734	0.893777	0.690003
1	2	0.856153	0.927126	0.778912	0.846580	0.914173	0.720043
2	3	0.821490	0.887097	0.745763	0.810313	0.892595	0.694669
3	4	0.859619	0.911538	0.803390	0.854054	0.923416	0.668841
4	5	0.838821	0.876866	0.796610	0.834813	0.912405	0.623642

```
results_df.to_csv("Mol2VecSVM_fold_results_with_validation.csv", index=False)

summary_rows = []
for metric in ["accuracy", "precision", "recall", "f1", "roc_auc"]:
    mean_val = results_df[metric].mean()
    std_val = results_df[metric].std(ddof=1)
    var_val = results_df[metric].var(ddof=1)
    summary_rows.append({
        "Metric": metric.upper(),
        "Mean": mean_val,
        "Std": std_val,
        "Variance": var_val,
        "Formatted": f"{mean_val:.4f} ± {std_val:.4f}"
    })

summary_df = pd.DataFrame(summary_rows)
summary_df.to_csv("Mol2VecSVM_summary_results_with_validation.csv", index=False)

print("Fold bazlı sonuçlar:")
display(results_df)

print("\nÖzet sonuçlar:")
display(summary_df)
```

Fold bazlı sonuçlar:

	fold	accuracy	precision	recall	f1	roc_auc	threshold
0	1	0.826690	0.904167	0.738095	0.812734	0.893777	0.690003
1	2	0.856153	0.927126	0.778912	0.846580	0.914173	0.720043
2	3	0.821490	0.887097	0.745763	0.810313	0.892595	0.694669
3	4	0.859619	0.911538	0.803390	0.854054	0.923416	0.668841
4	5	0.838821	0.876866	0.796610	0.834813	0.912405	0.623642

Özet sonuçlar:

	Metric	Mean	Std	Variance	Formatted
0	ACCURACY	0.840555	0.017069	0.000291	0.8406 ± 0.0171
1	PRECISION	0.901359	0.019867	0.000395	0.9014 ± 0.0199
2	RECALL	0.772554	0.029475	0.000869	0.7726 ± 0.0295
3	F1	0.831699	0.019672	0.000387	0.8317 ± 0.0197
4	ROC_AUC	0.907273	0.013529	0.000183	0.9073 ± 0.0135

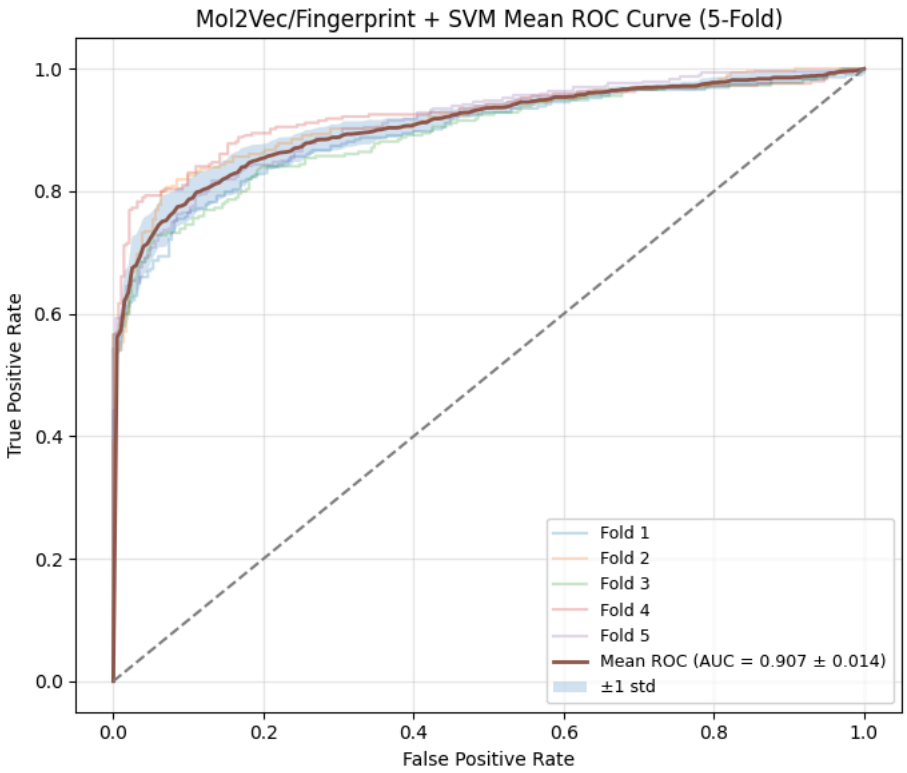
```
mean_fpr = np.linspace(0, 1, 200)
interp_tprs = []
fold_aucs = []

for fpr_i, tpr_i in roc_data:
    interp_tpr = np.interp(mean_fpr, fpr_i, tpr_i)
    interp_tpr[0] = 0.0
    interp_tprs.append(interp_tpr)
    fold_aucs.append(auc(fpr_i, tpr_i))

mean_tpr = np.mean(interp_tprs, axis=0)
std_tpr = np.std(interp_tprs, axis=0)
mean_tpr[-1] = 1.0
mean_auc = np.mean(fold_aucs)
std_auc = np.std(fold_aucs, ddof=1)

plt.figure(figsize=(7, 6))
for i, (fpr_i, tpr_i) in enumerate(roc_data, start=1):
    plt.plot(fpr_i, tpr_i, alpha=0.25, label=f"Fold {i}")

plt.plot(mean_fpr, mean_tpr, linewidth=2, label=f"Mean ROC (AUC = {mean_auc:.3f} ± {std_auc:.3f})")
plt.fill_between(mean_fpr, np.maximum(mean_tpr - std_tpr, 0), np.minimum(mean_tpr + std_tpr, 1), alpha=0.2, label="±1 std")
plt.plot([0, 1], [0, 1], linestyle="--", color="gray")
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("Mol2Vec/Fingerprint + SVM Mean ROC Curve (5-Fold)")
plt.legend(loc="lower right", fontsize=9)
plt.grid(alpha=0.3)
plt.tight_layout()
plt.savefig("Mol2VecSVM_mean_ROC_curve.png", dpi=300, bbox_inches="tight")
plt.show()
```



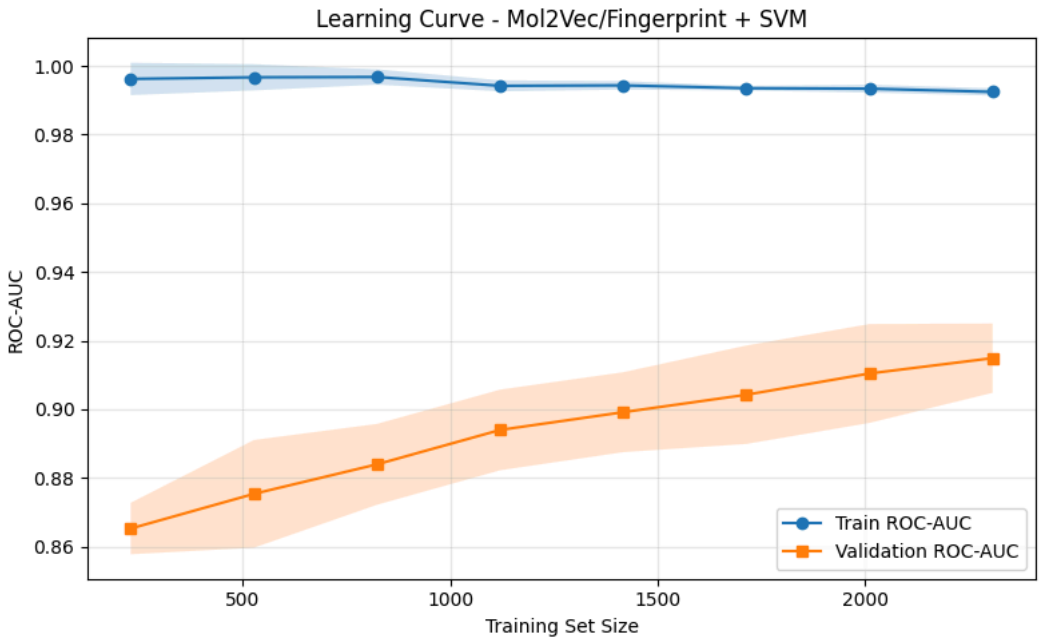

```
pipeline = Pipeline([
    ("scaler", StandardScaler()),
    ("svc", SVC(kernel="rbf", C=1.0, gamma="scale", probability=False, random_state=SEED))
])

cv = StratifiedKFold(n_splits=N_SPLITS, shuffle=True, random_state=SEED)
```

```
# Learning curve
train_sizes, train_scores, valid_scores = learning_curve(
    estimator=pipeline,
    X=X,
    y=y,
    train_sizes=np.linspace(0.1, 1.0, 8),
    cv=cv,
    scoring="roc_auc",
    n_jobs=-1,
    shuffle=True,
    random_state=SEED
)

train_mean = train_scores.mean(axis=1)
train_std = train_scores.std(axis=1, ddof=1)
valid_mean = valid_scores.mean(axis=1)
valid_std = valid_scores.std(axis=1, ddof=1)

plt.figure(figsize=(8, 5))
plt.plot(train_sizes, train_mean, marker="o", label="Train ROC-AUC")
plt.fill_between(train_sizes, train_mean - train_std, train_mean + train_std, alpha=0.2)
plt.plot(train_sizes, valid_mean, marker="s", label="Validation ROC-AUC")
plt.fill_between(train_sizes, valid_mean - valid_std, valid_mean + valid_std, alpha=0.2)
plt.xlabel("Training Set Size")
plt.ylabel("ROC-AUC")
plt.title("Learning Curve - Mol2Vec/Fingerprint + SVM")
plt.legend()
plt.grid(alpha=0.3)
plt.tight_layout()
plt.savefig("Mol2VecSVM_learning_curve.png", dpi=300, bbox_inches="tight")
plt.show()
```




```
# Validation curve (C parametresi)
param_range = np.logspace(-2, 2, 7)

train_scores_v, valid_scores_v = validation_curve(
    estimator=pipeline,
    X=X,
    y=y,
    param_name="svc__C",
    param_range=param_range,
    cv=cv,
    scoring="roc_auc",
    n_jobs=-1
)

train_mean_v = train_scores_v.mean(axis=1)
train_std_v = train_scores_v.std(axis=1, ddof=1)
valid_mean_v = valid_scores_v.mean(axis=1)
valid_std_v = valid_scores_v.std(axis=1, ddof=1)

val_curve_df = pd.DataFrame({
    "C": param_range,
    "Train_Mean_ROC_AUC": train_mean_v,
    "Train_Std": train_std_v,
    "Validation_Mean_ROC_AUC": valid_mean_v,
    "Validation_Std": valid_std_v
})
val_curve_df.to_csv("Mol2VecSVM_validation_curve_C.csv", index=False)
display(val_curve_df)

plt.figure(figsize=(8, 5))
plt.semilogx(param_range, train_mean_v, marker="o", label="Train ROC-AUC")
plt.fill_between(param_range, train_mean_v - train_std_v, train_mean_v + train_std_v, alpha=0.2)
plt.semilogx(param_range, valid_mean_v, marker="s", label="Validation ROC-AUC")
plt.fill_between(param_range, valid_mean_v - valid_std_v, valid_mean_v + valid_std_v, alpha=0.2)

best_idx = np.argmax(valid_mean_v)
best_c = param_range[best_idx]
best_val_auc = valid_mean_v[best_idx]
plt.axvline(best_c, linestyle="--", color="gray", alpha=0.7, label=f"Best C = {best_c:.3g}")

plt.xlabel("C Parameter (log scale)")
plt.ylabel("ROC-AUC")
plt.title("Validation Curve - Mol2Vec/Fingerprint + SVM")
plt.legend()
plt.grid(alpha=0.3)
plt.tight_layout()
plt.savefig("Mol2VecSVM_validation_curve_C.png", dpi=300, bbox_inches="tight")
plt.show()

print(f"Best C according to validation mean ROC-AUC: {best_c:.4g}")
print(f"Best validation ROC-AUC: {best_val_auc:.4f}")
```

	C	Train_Mean_ROC_AUC	Train_Std	Validation_Mean_ROC_AUC	Validation_Std
0	0.010000	0.798938	0.003021	0.744455	0.031264
1	0.046416	0.823505	0.003708	0.765202	0.025748
2	0.215443	0.938613	0.002136	0.867660	0.017138
3	1.000000	0.992435	0.001054	0.914913	0.010082
4	4.641589	0.999835	0.000256	0.919531	0.006535
5	21.544347	0.999626	0.000266	0.919283	0.006989
6	100.000000	0.999513	0.000283	0.919341	0.007084

